

Deep Learning - Theory and Practice

Deep Neural Networks

12-03-2020

<http://leap.ee.iisc.ac.in/sriram/teaching/DL20/>

deeplearning.cce2020@gmail.com



Logistic Regression

- ❖ 2- class logistic regression

$$p(C_1|\phi) = y(\phi) = \sigma(\mathbf{w}^T \phi)$$

- ❖ Maximum likelihood solution

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \phi_n$$

- ❖ K-class logistic regression

$$p(C_k|\phi) = y_k(\phi) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$$

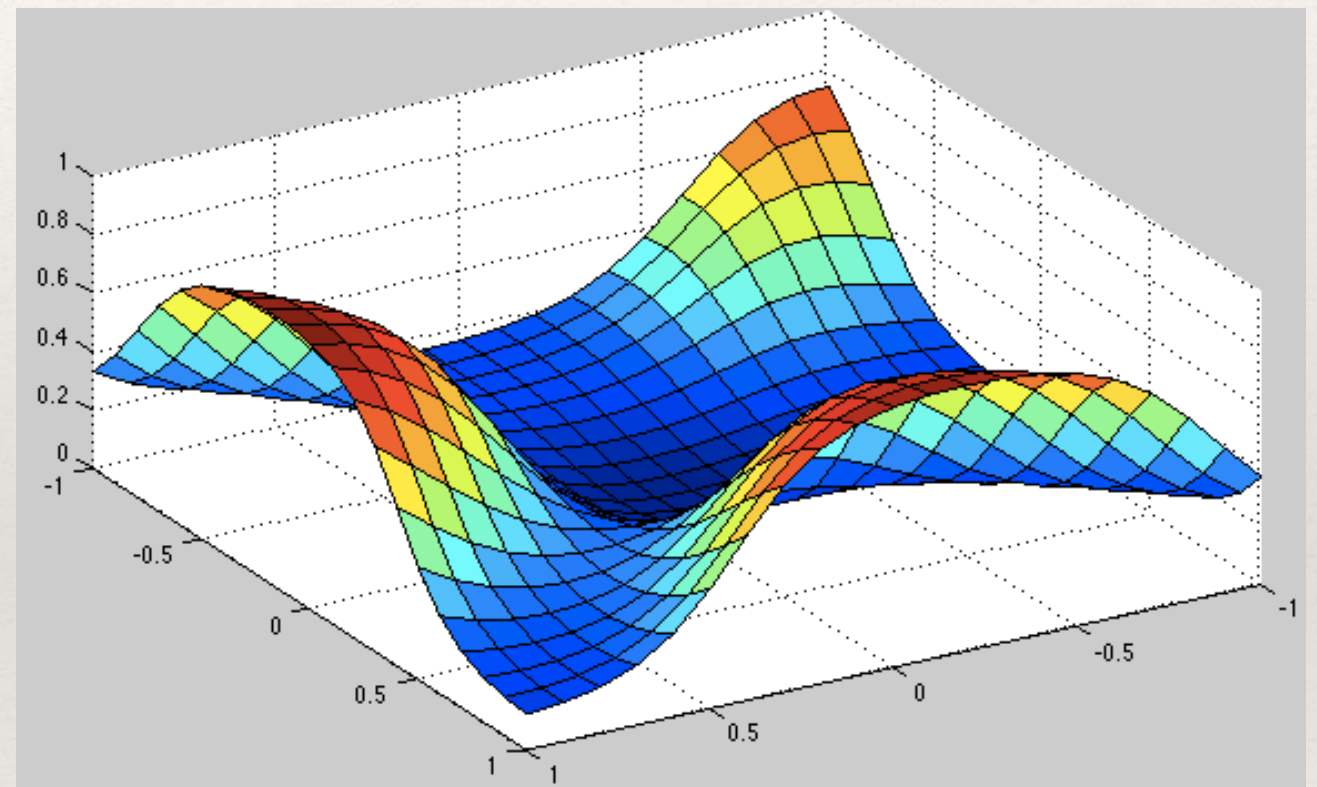
- ❖ Maximum likelihood solution

$$a_k = \mathbf{w}_k^T \phi.$$

$$\nabla_{\mathbf{w}_j} E(\mathbf{w}_1, \dots, \mathbf{w}_K) = \sum_{n=1}^N (y_{nj} - t_{nj}) \phi_n$$

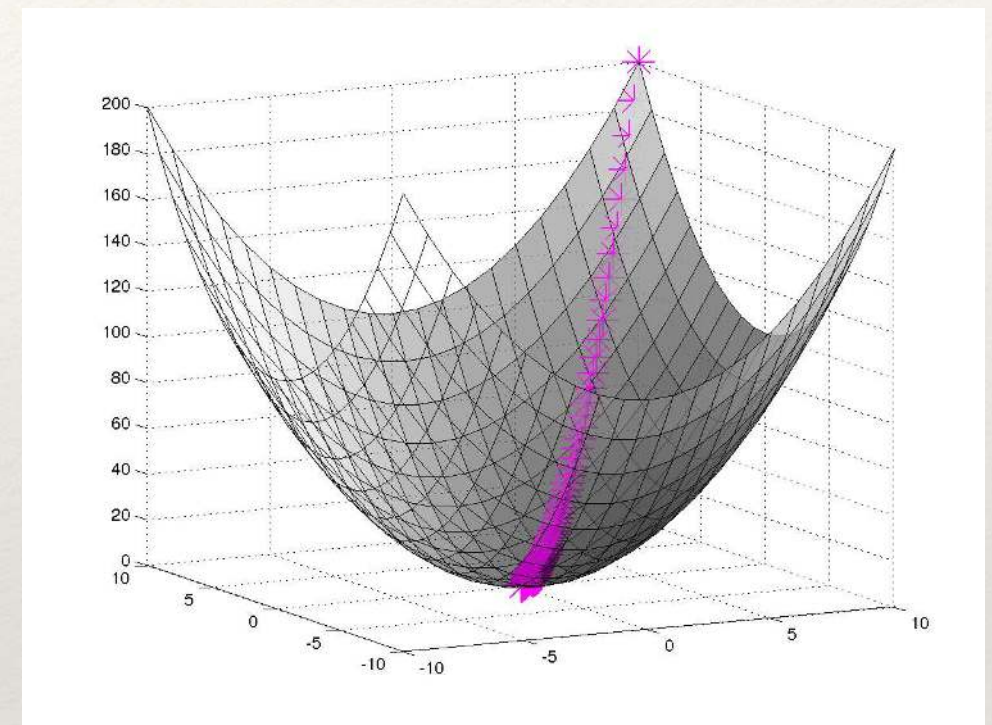
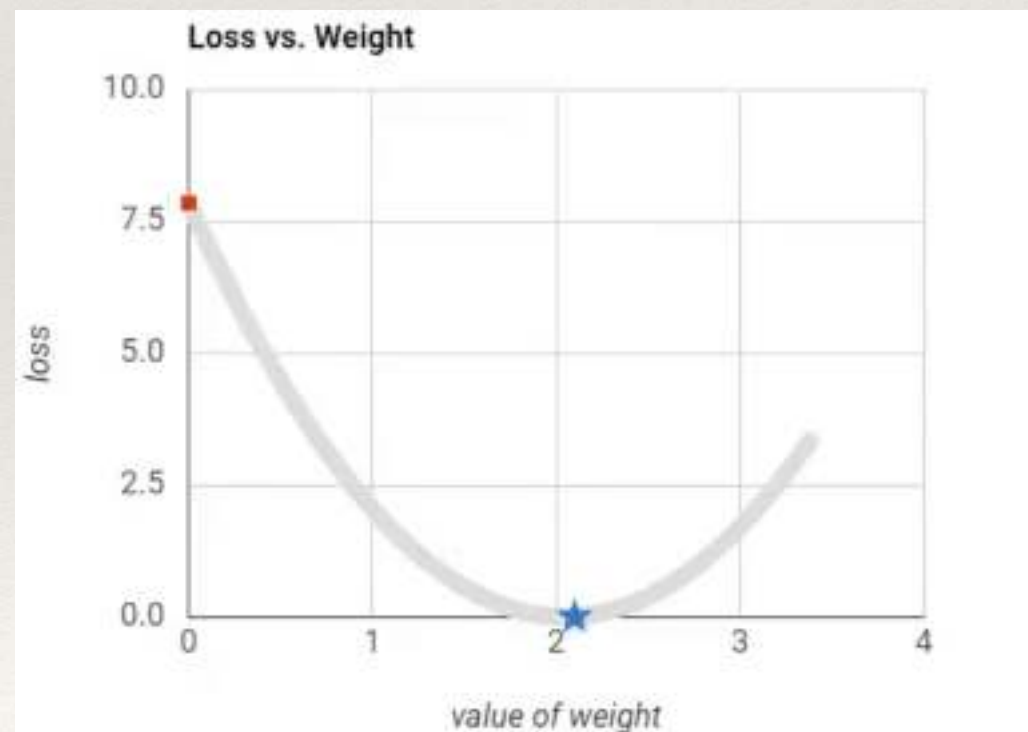
Typical Error Surfaces

Typical Error Surface as a function of parameters (weights and biases)

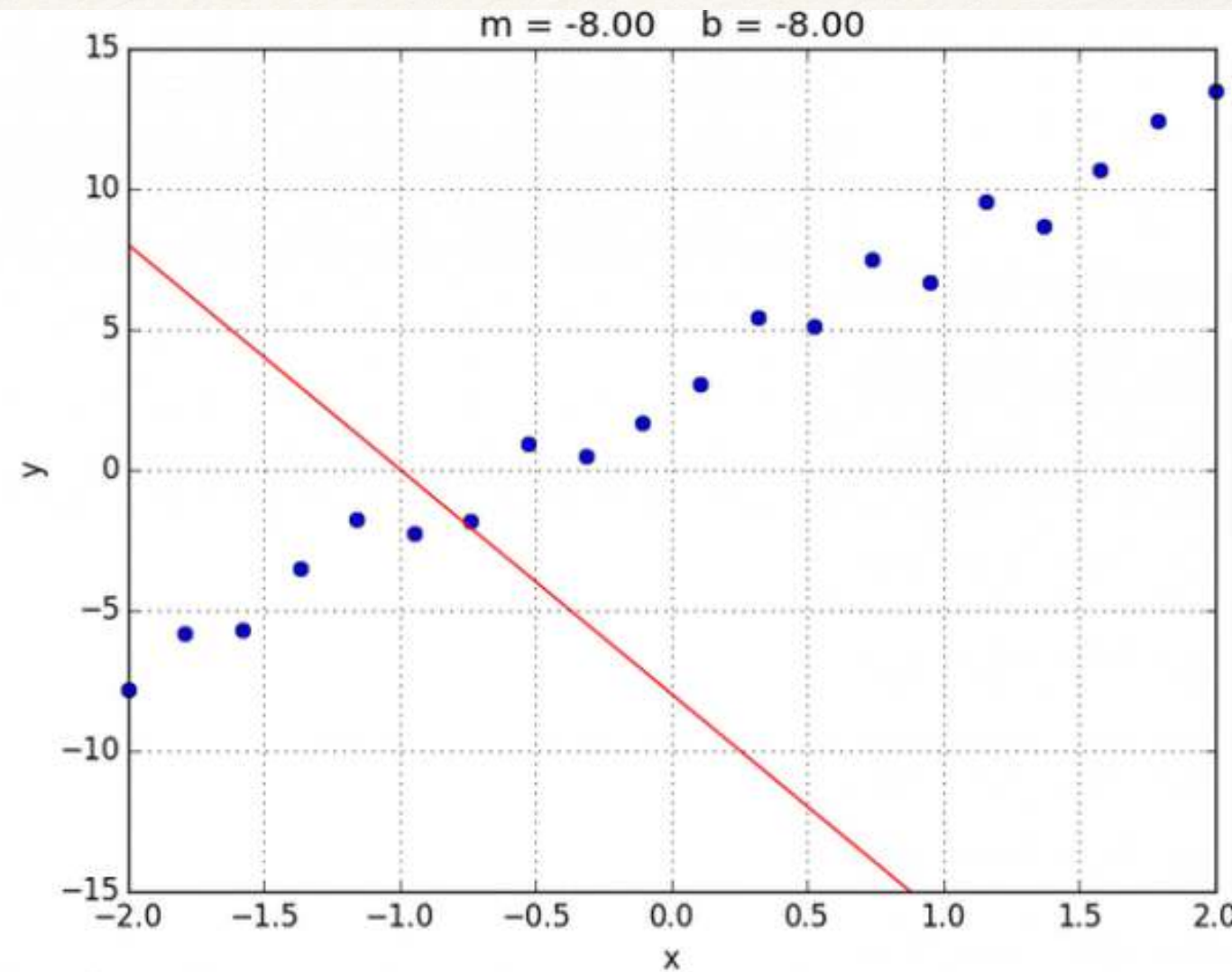
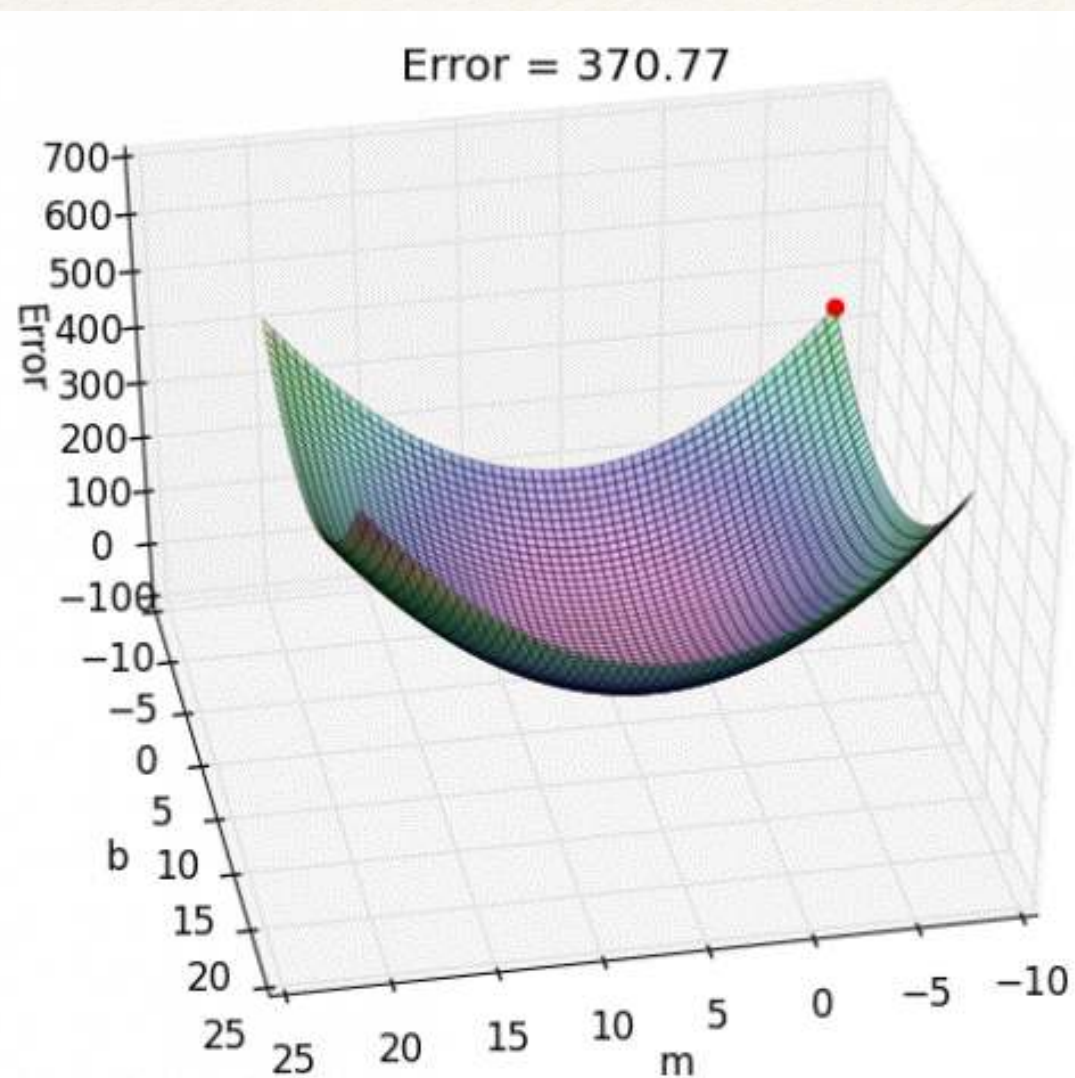


Learning with Gradient Descent

Error surface close to a local

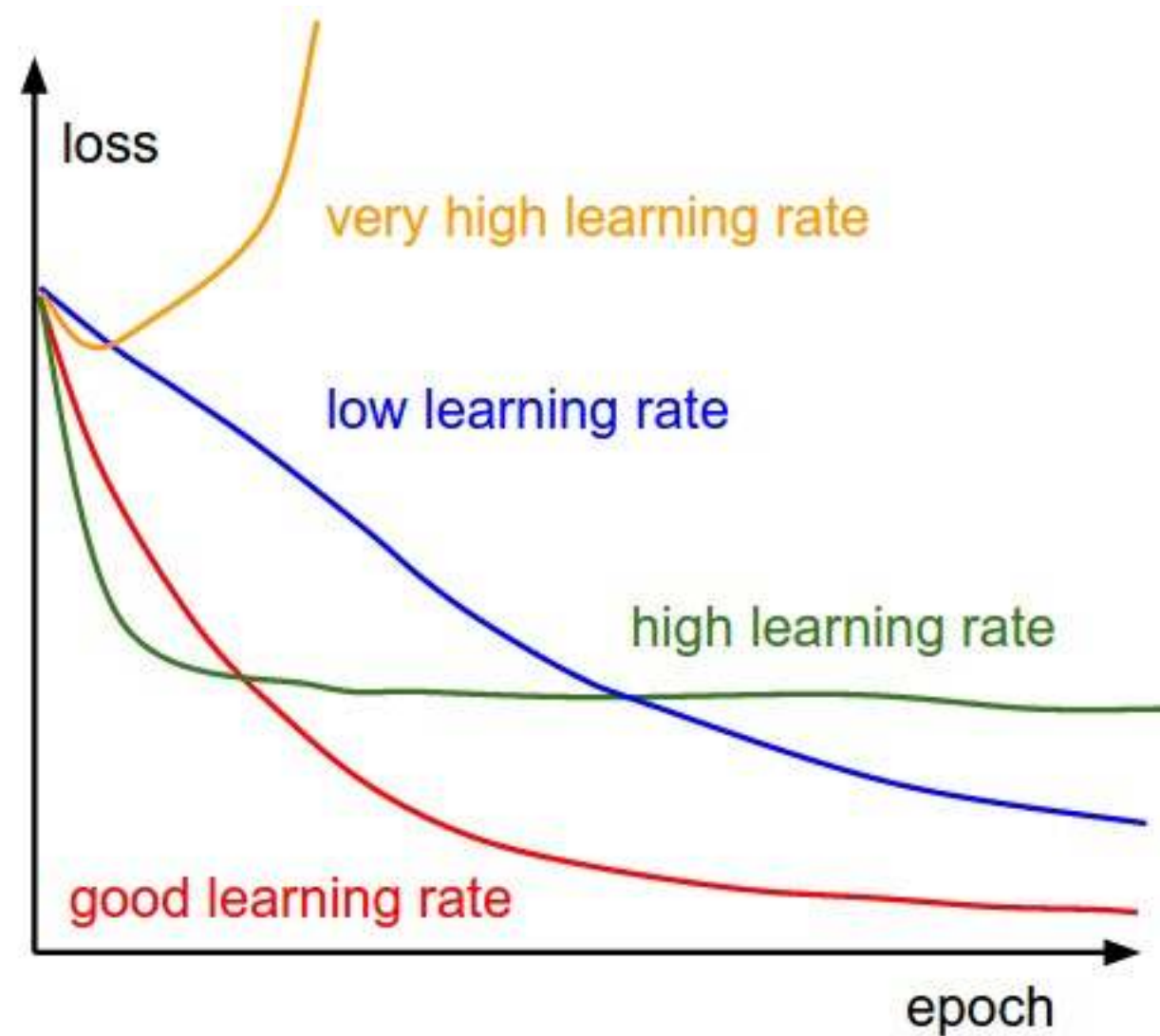


Learning Using Gradient Descent

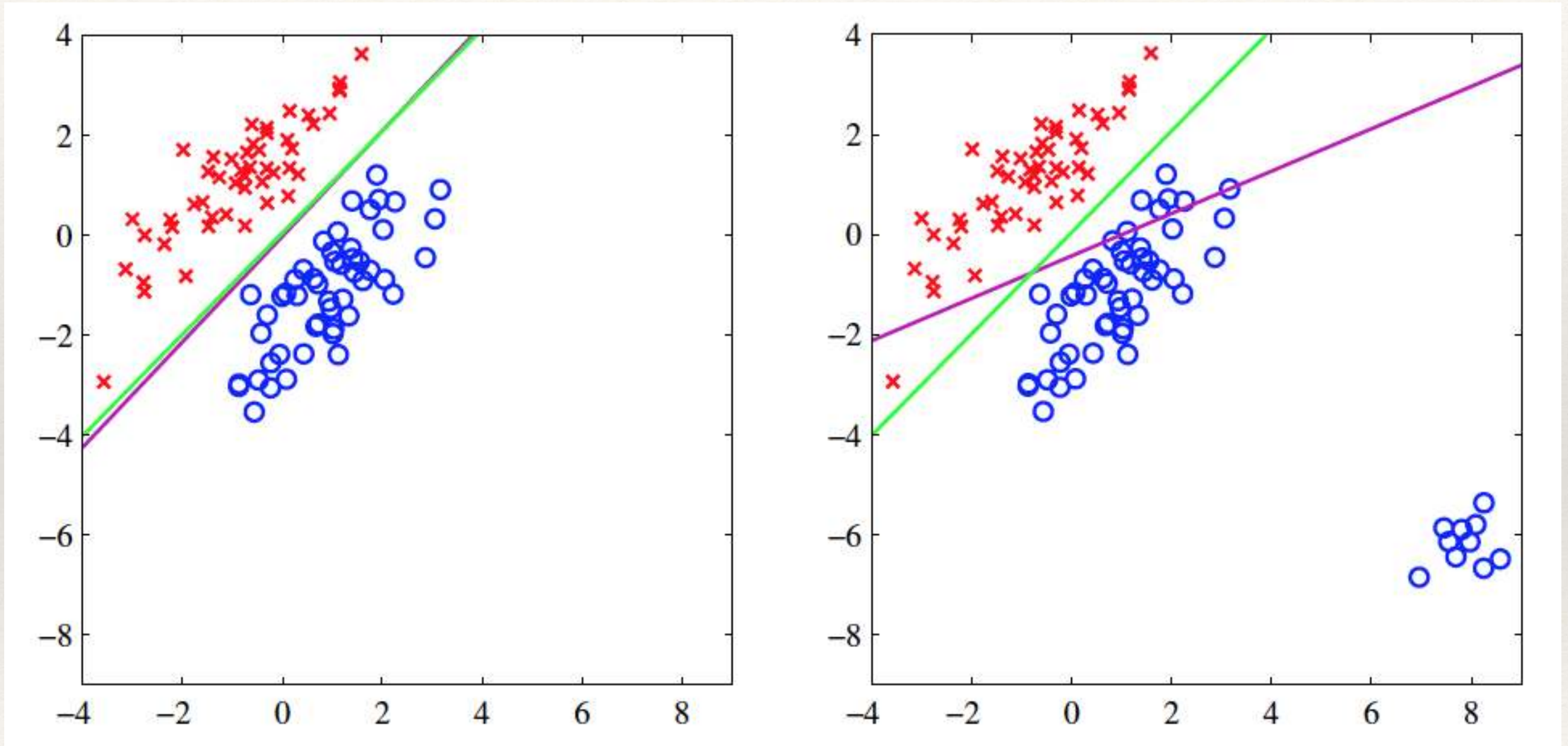


Parameter Learning

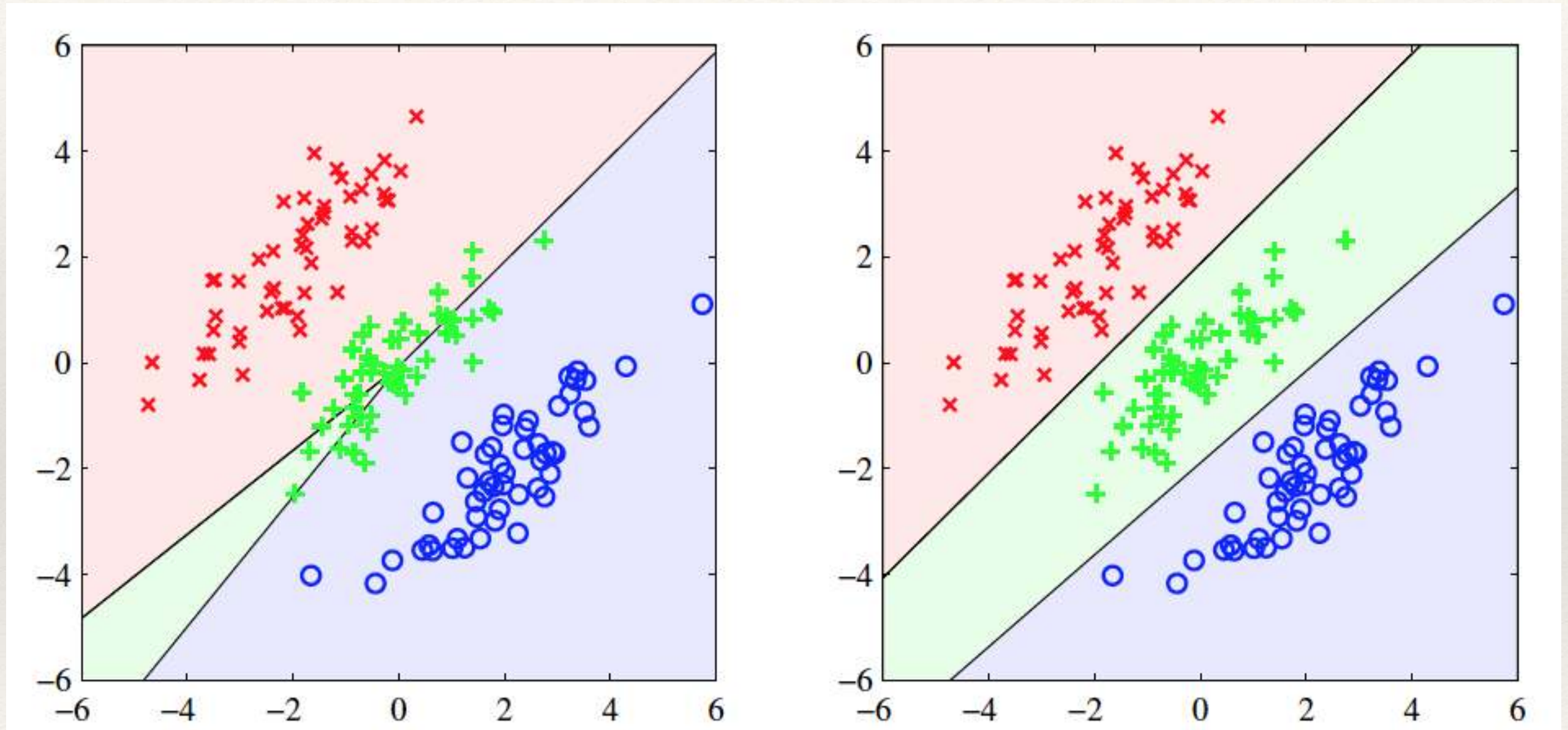
- Solving a non-convex optimization.
- Iterative solution.
- Depends on the initialization.
- Convergence to a local optima.
- Judicious choice of learning rate



Least Squares versus Logistic Regression



Least Squares versus Logistic Regression

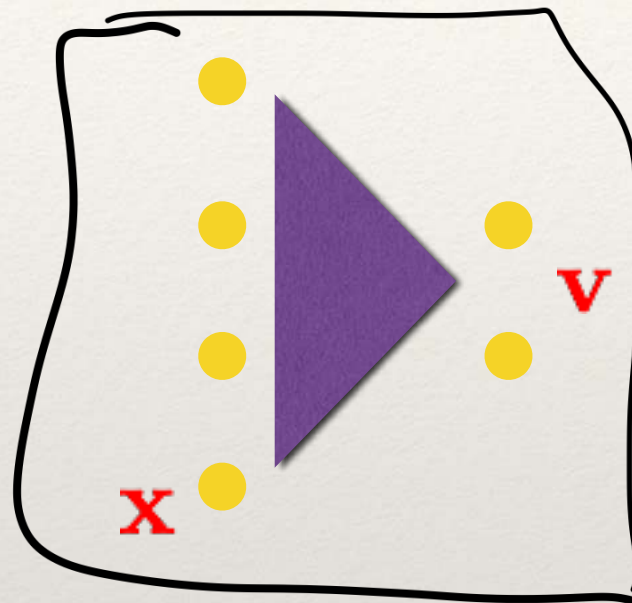


Bishop - PRML book (Chap 4)

Neural Networks

Perceptron Algorithm

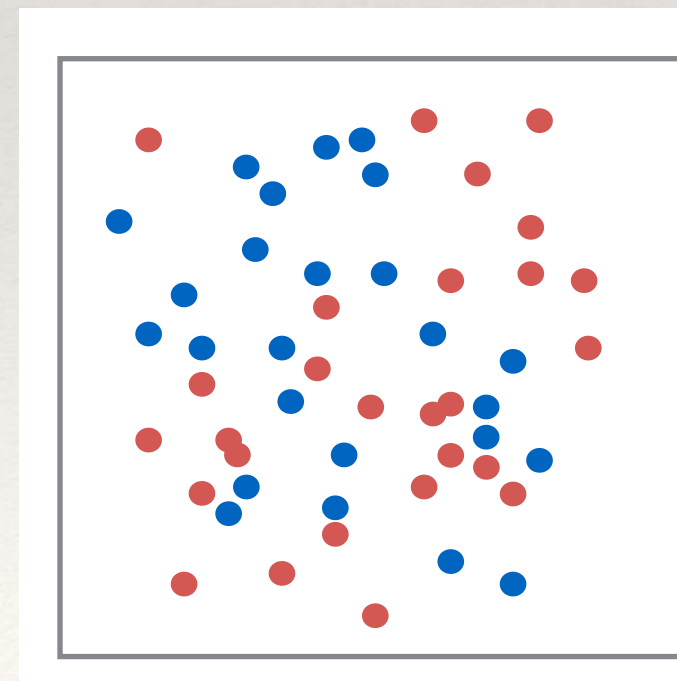
Perceptron Model [McCulloch, 1943, Rosenblatt, 1957]



Similar to the
logistic
regression

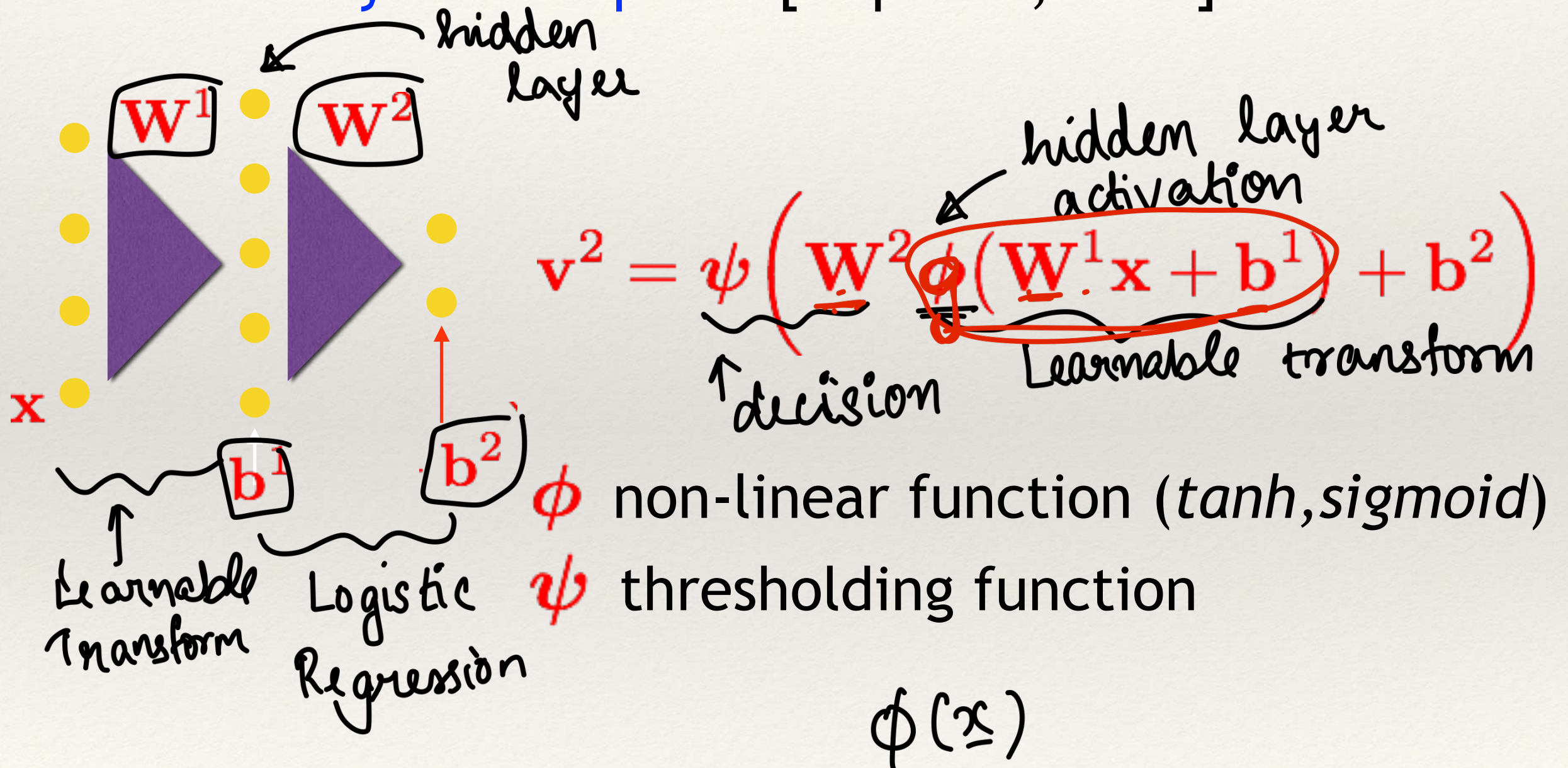
Targets are binary classes $[-1, 1]$

What if the data is not
linearly separable



Multi-layer Perceptron (MLP)

Multi-layer Perceptron [Hopfield, 1982]



x $\xrightarrow{\text{learnable}}$ $\phi(x)$ $\xrightarrow{\text{vector valued}}$

$\phi(x)$ $\xrightarrow{\text{optimization done in a logistic regression}}$ y [outputs]

[cost function - CE
Regression - linear - M.S.E]

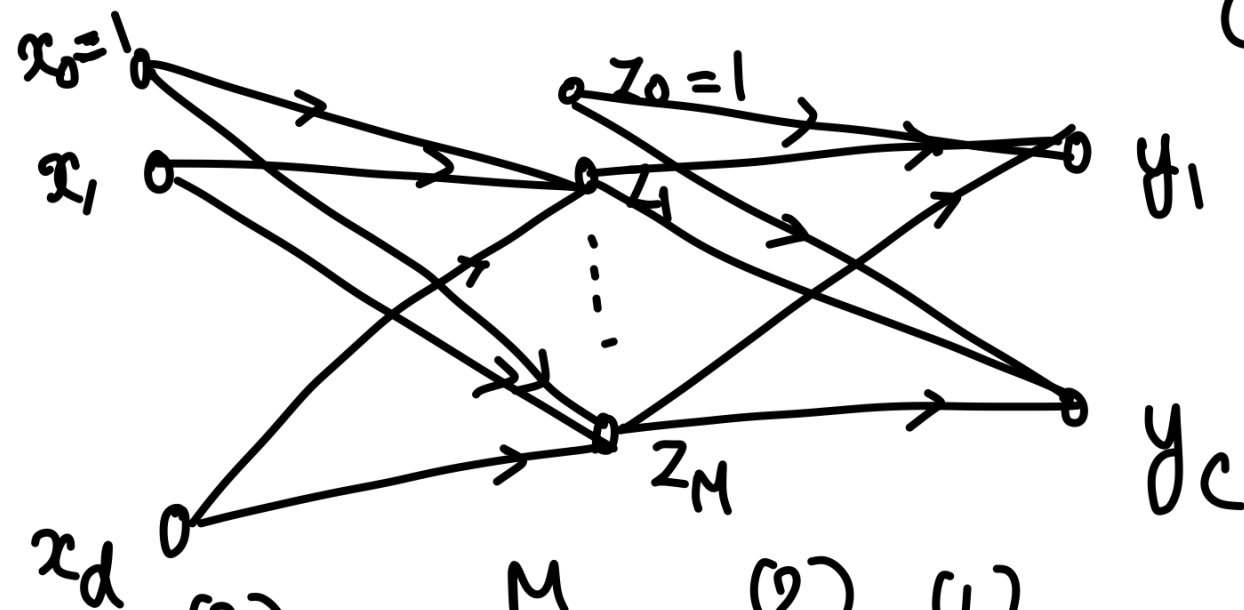
Learning [S.G.D]

$\rightarrow$ Previously only w^2, b^2 were the variables

$\rightarrow \{w^1, b^1, w^2, b^2\}$ variables in this model.

Back propagation

[Neural Networks, C. Bishop
CHAP - 4]



$$a_k^{(2)} = \sum_{j=0}^M w_{kj}^{(2)} z_j^{(1)}$$

$$y_k = \tilde{g}(a_k^{(2)})$$

t_k^n = target for k^{th} node for $\underline{x}_n \{ \underline{x}_1, \underline{x}_2 \dots \underline{x}_n \dots \}_N$
 $\{ t_1, t_2 \dots t_N \}$

$$E = \sum_n E^n$$

; All functions in the forward pass are differentiable.

Forward Direction

$$a_j^{(1)} = \sum_{i=0}^d w_{ji}^{(1)} x_i$$

$j = 1 \dots M$

$$z_j^{(1)} = g(a_j)$$

non-linear activation.

Gradient descent :

$$\frac{\partial E^n}{\partial w_{ji}^l}$$

$$l = \{1, 2\}$$

Let

$$\delta_j^l = \frac{\partial E^n}{\partial a_j^l}$$

:

$$\frac{\partial a^l}{\partial w_{ji}^l}$$

$$= z_i^{l-1}$$

$$z_i^0 = x_i$$

$$\frac{\partial E^n}{\partial w_{ji}^l}$$

$$= \frac{\partial E^n}{\partial a_j^l}$$

$$\cdot \frac{\partial a_j^l}{\partial w_{ji}^l}$$

$$= \delta_j^l \cdot z_i^{l-1}$$

Deep networks > 1 hidden layers

$$a_j^l = \sum_i w_{ji}^{l-1} z_i^{l-1}$$

$$z_j^l = g(a_j^l)$$

For $l = L$ { L is number of layers }

$$\delta_k^L$$

$$= \frac{\partial E^n}{\partial a_k^L}$$

$$= g'(a_k^L) \cdot \frac{\partial E^n}{\partial y_k^n}$$

$$\frac{\partial E^n}{\partial w_{ji}^L}$$

Now, for any layer $L, \dots, l, \dots, 2$

Example

$$E^n = - \sum_{k=1}^c t_k^n \log y_k^n + \lambda R(w)$$

δ_j^{l-1}
 $\uparrow$
 $\frac{\partial \mathcal{E}^n}{\partial a_j^{l-1}}$

δ_k^l
 $\leftarrow$
 $\frac{\partial \mathcal{E}^n}{\partial z_j^{l-1}} \cdot \frac{\partial z_j^{l-1}}{\partial a_j^{l-1}}$

$$\begin{aligned}
 \delta_j^{l-1} &= \frac{\partial \mathcal{E}^n}{\partial a_j^{l-1}} = \frac{\partial \mathcal{E}^n}{\partial z_j^{l-1}} \cdot \frac{\partial z_j^{l-1}}{\partial a_j^{l-1}} \\
 &= \sum_k \frac{\partial \mathcal{E}^n}{\partial a_k^l} \cdot \frac{\partial a_k^l}{\partial z_j^{l-1}} \cdot g'(a_j^{l-1}) \\
 &= \sum_k \delta_k^l w_{kj}^{(l)} g'(a_j^{l-1})
 \end{aligned}$$

$z_j^{l-1} = g(a_j^{l-1})$
 $a_k^l = \sum_j w_{kj}^{(l)} z_j^{l-1}$
 $\frac{\partial \mathcal{E}^n}{\partial a_k^l}$

Backward pass.

Update equation
t-iteration

$$\{w_{kj}^l\}^t = \{w_{kj}^l\}^{t-1} - \eta \frac{\partial \mathcal{E}}{\partial w_{kj}^l} \Big|_{\{w_{kj}^l = w_{kj}^l\}^{t-1}}$$

Summary

↓ Initial set of weights.

(1) $\underline{x}^n \rightarrow$ find activations $a_k^l, z_k^l \forall l$.
$$z_i^0 = x_i \quad ; \quad z_k^L = y_k.$$

(2) Evaluate δ at L , then apply recursion
$$\delta_k^l \rightarrow \delta_j^{l-1} \quad \underline{l = L, L-1, \dots, 2.}$$

(3) Compute derivatives of weight and update.

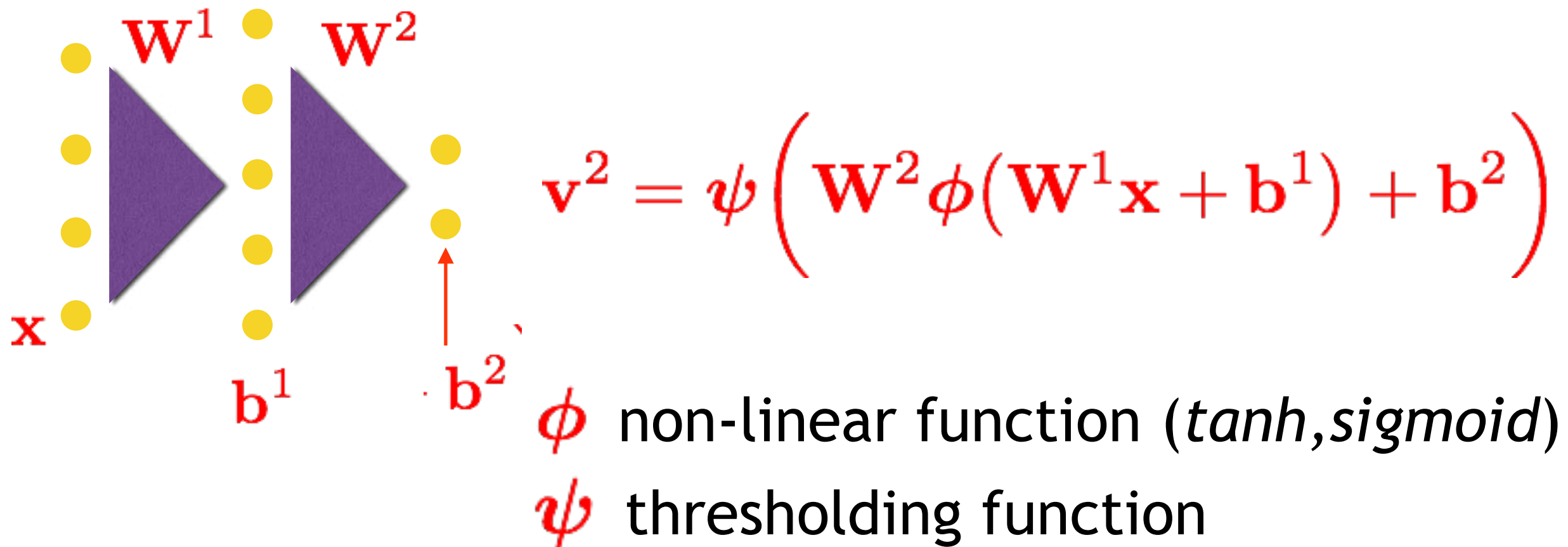
→ Repeat until convergence

↑ Fixed number of epochs

↑ Error in validation
reaching saturation

Neural Networks

Multi-layer Perceptron [Hopfield, 1982]

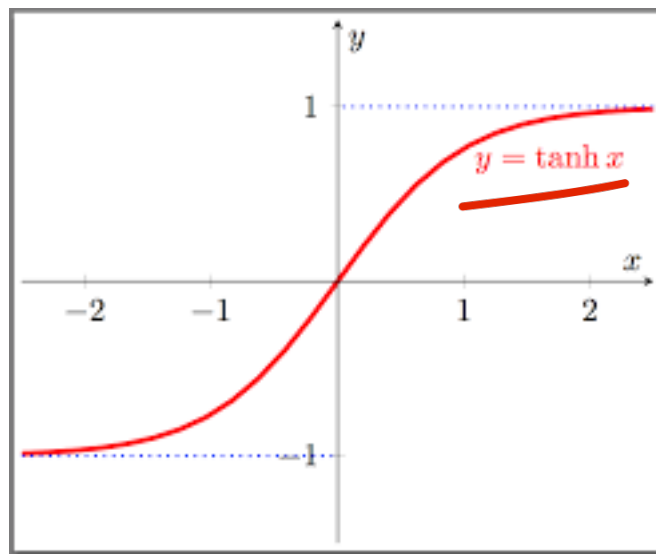


- Useful for classifying non-linear data boundaries - non-linear class separation can be realized given enough data.

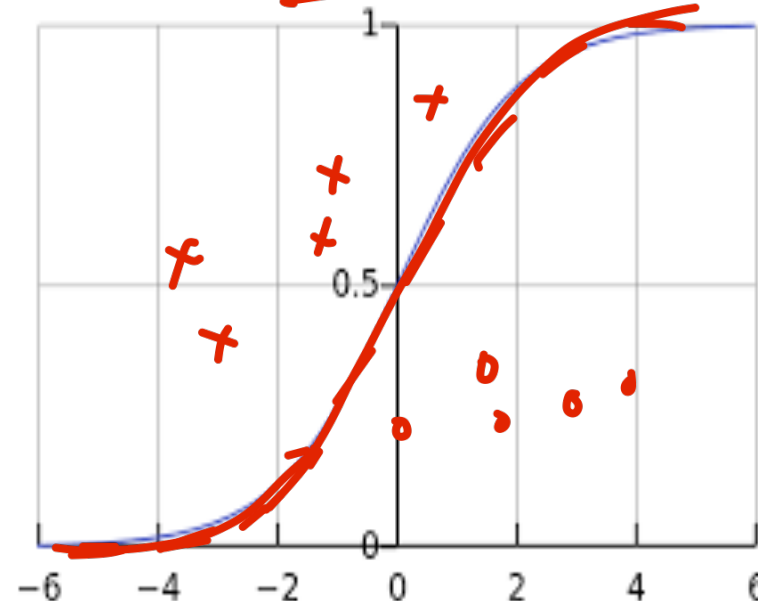
Neural Networks

Types of Non-linearities $\phi(g)$

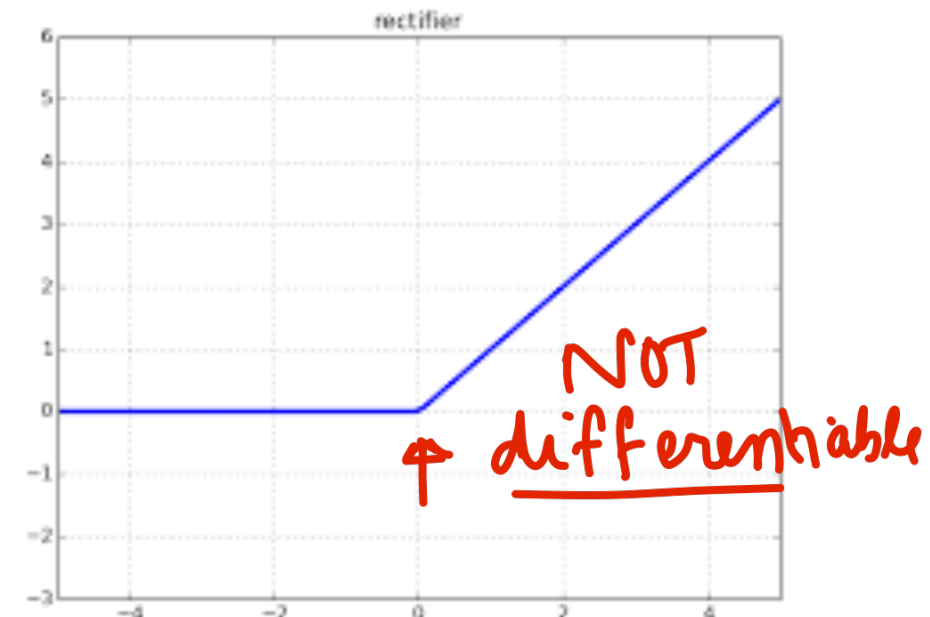
tanh



sigmoid



ReLU



Cost-Function

Mean Square Error

$$\underline{E J_{MSE}} = \sum_{i=1}^M ||\mathbf{v}_i - \mathbf{y}_i||^2$$

Cross Entropy

$$E J_{CE} = - \sum_{i=1}^M \mathbf{y}_i^T \log(\mathbf{v}_i)$$

$\mathbf{z}_k^l = \mathbf{a}_k^{l-1}$ if $\mathbf{a}_k^{l-1} > 0$
 $= 0$ otherwise
 $\mathbf{a}_k^{l-1} = \mathbf{w}_{k-1}^{(l-1)} \mathbf{z}_{k-1}^{(l-2)}$

$\mathbf{y}_i$ are the desired outputs

Learning Posterior Probabilities with NNs

Choice of target function ψ

- Softmax function for classification

$$\psi(v_i) = \frac{e^{v_i}}{\sum_i e^{v_i}}$$

- Softmax produces positive values that sum to 1
- Allows the interpretation of outputs as posterior probabilities

Need For Deep Networks

Modeling complex real world data like **speech, image, text**

- Single hidden layer networks are **too restrictive**.
- Needs large number of units in the hidden layer and trained with large amounts of data.
- Not generalizable enough.

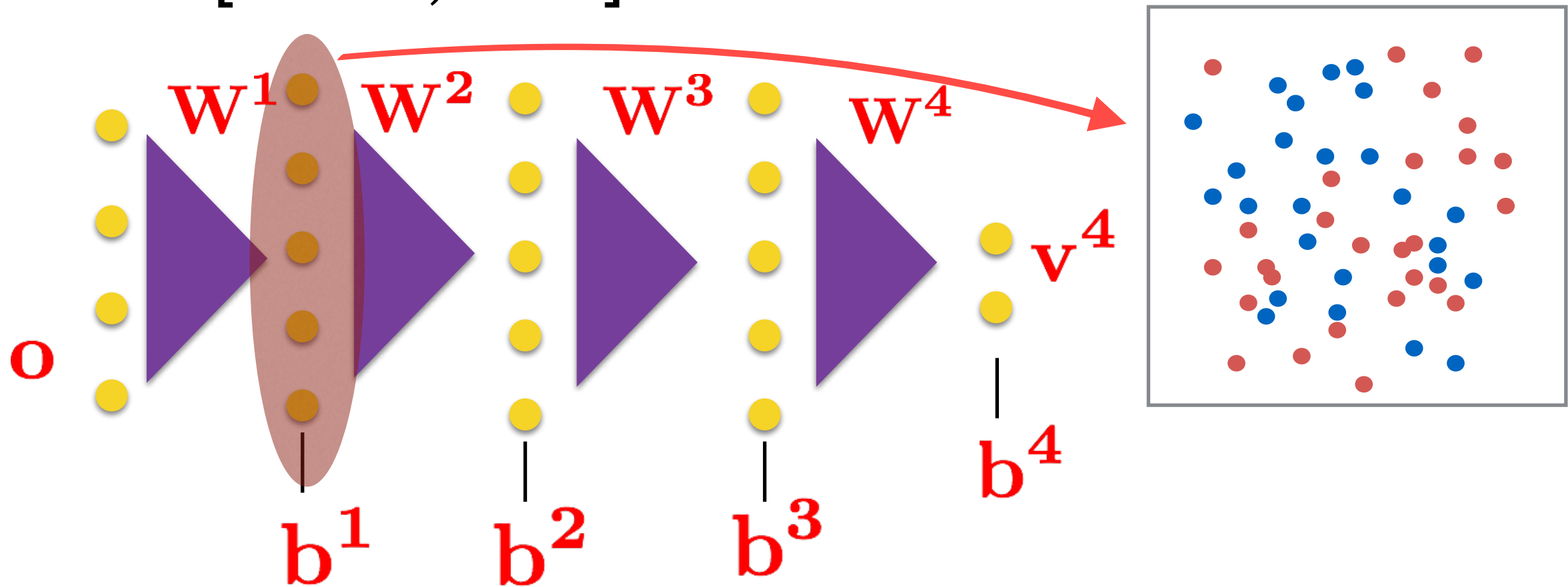
Networks with **multiple hidden layers** - deep networks

(Open questions till 2005)

- Are these networks trainable ?
- How can we initialize such networks ?
- Will these generalize well or over train ?

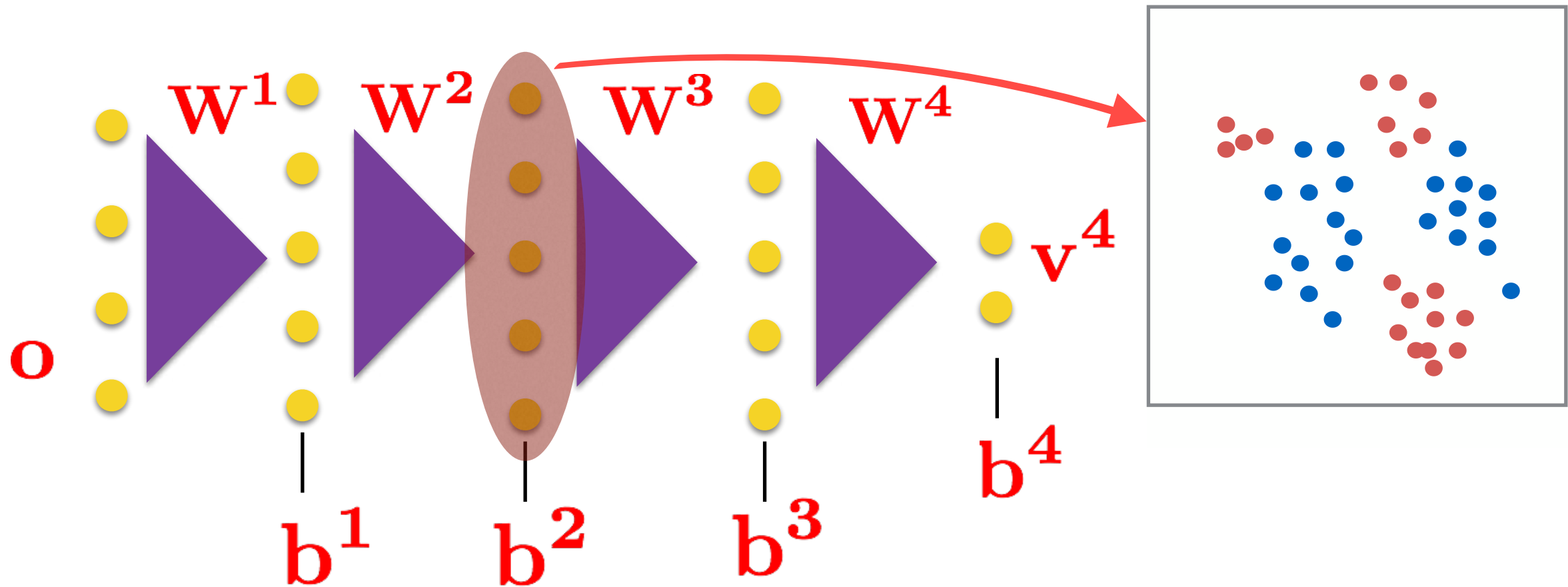
Deep Networks Intuition

Neural networks with multiple hidden layers - Deep networks [Hinton, 2006]



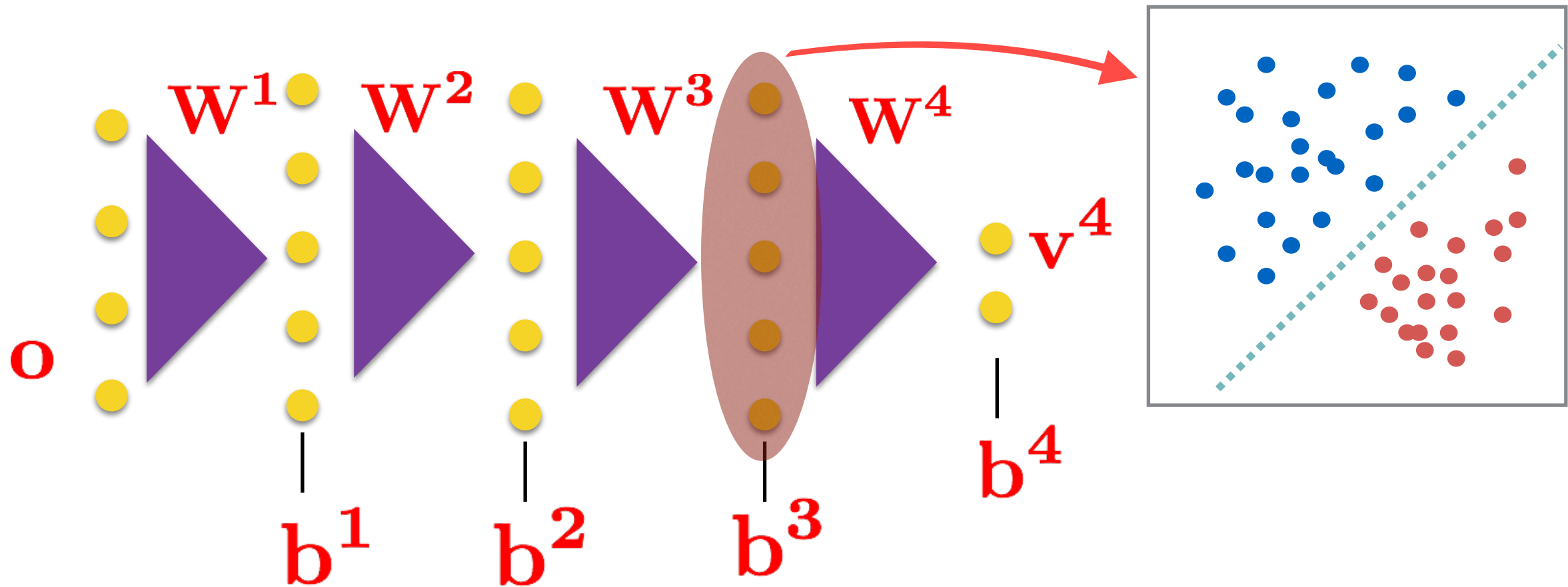
Deep Networks Intuition

Neural networks with multiple hidden layers - Deep networks



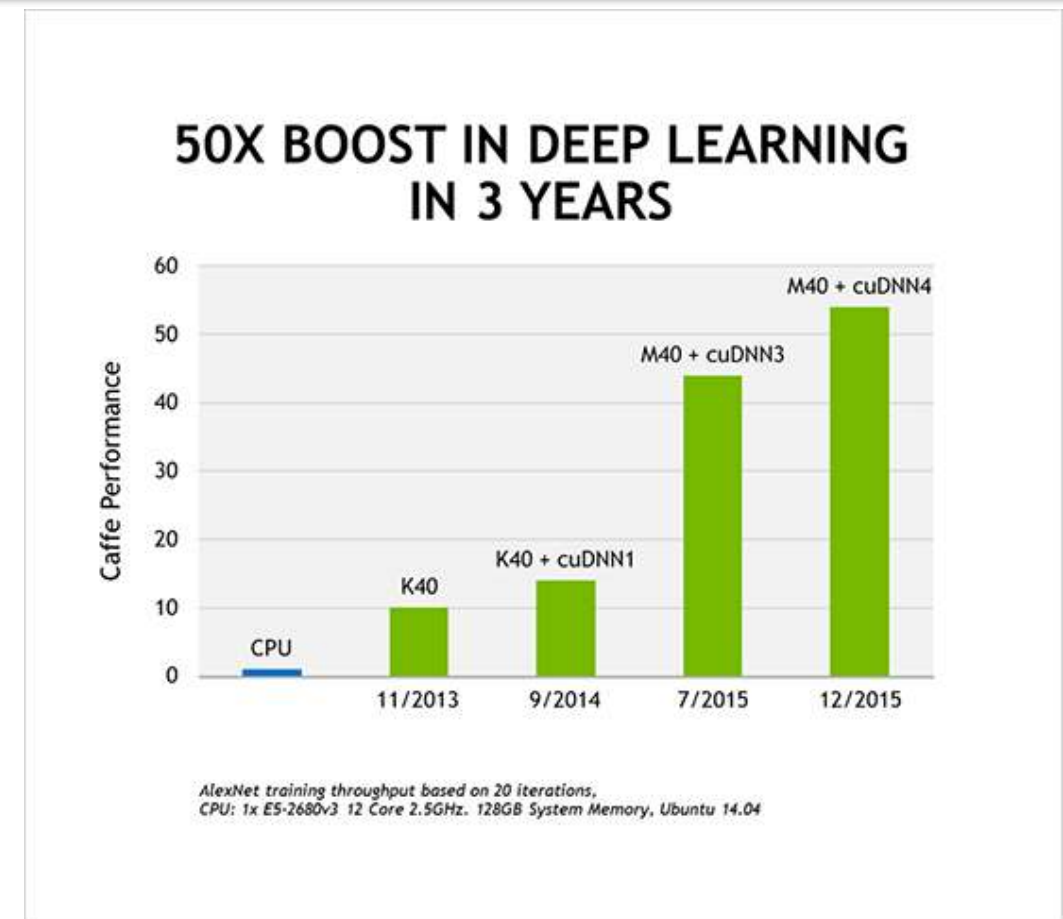
Deep Networks Intuition

Neural networks with multiple hidden layers - Deep networks



Deep networks perform **hierarchical data abstractions** which enable the non-linear separation of complex data samples.

Deep Networks



- Are these networks trainable ?
 - Advances in computation and processing
 - **Graphical processing units** (GPUs) performing multiple parallel multiply accumulate operations.
 - Large amounts of supervised data sets